

Machine Learning — In-Depth EDA, Data Cleaning & Preprocessing

Practical workflow, decisions, formulas and Python patterns

1. Where EDA and Preprocessing Fit

The PDF's ML workflow places EDA before cleaning/preprocessing and then moves to feature engineering/selection, splitting, modeling and evaluation. This ordering matters because you need to understand the data before deciding how to transform it.

Problem → Data → **EDA** → **Cleaning** → **Preprocessing** → Feature Engineering/Selection → Split/Model → Evaluation

2. EDA — Step-by-Step

2.1 Inspect structure

Check shape, columns, head/tail, info and data types. Questions: What is one row? What does each feature mean? Which feature is the target?

2.2 Numerical summary

Use mean, median, standard deviation, min, max and quartiles. Compare mean vs median to notice skewness and unusual values.

2.3 Categorical summary

Use `value_counts()`. Check rare categories, spelling variations, unexpected labels and class imbalance.

2.4 Missingness

Calculate missing counts and percentages by column. Also inspect whether missingness is concentrated in certain groups or rows.

2.5 Duplicates

Check exact duplicate rows. Decide whether duplicates represent repeated records or legitimate repeated observations.

2.6 Data types

Verify that numeric data is numeric, dates are datetime-like, and categorical fields are represented consistently.

2.7 Univariate distributions

Use histograms/density-style plots for numerical features and bar charts for categorical features.

2.8 Outliers

Use boxplots, IQR or z-scores to identify unusual values. Then investigate whether they are errors or genuine observations.

2.9 Bivariate relationships

Use scatter plots for numerical-numerical relationships and grouped summaries/plots for numerical-categorical relationships.

2.10 Correlation

Inspect correlations among numerical variables. High correlation can indicate redundancy, but correlation is not proof of causation.

2.11 Target exploration

For regression, inspect target distribution and relationships with predictors. For classification, inspect class counts and predictor relationships.

Core EDA Python template

```
# Structure
df.head()
df.tail()
df.shape
df.info()
df.dtypes

# Summary statistics
df.describe()
df.describe(include="all")

# Categorical frequencies
df["category"].value_counts(dropna=False)

# Missing values
df.isna().sum()
(df.isna().mean() * 100).sort_values(ascending=False)

# Duplicates
df.duplicated().sum()

# Correlation
df.select_dtypes("number").corr()
```

3. Missing Values — Decision Framework

Missing values are a data-quality issue, but there is no universal rule such as 'always drop nulls'. The treatment depends on the amount, meaning and type of missingness.

Situation	Possible action	Reasoning
Very few affected rows	Drop rows	Can be reasonable if information loss is negligible.
Numerical feature with moderate missingness	Mean/median imputation	Simple baseline; median is often less affected by extreme values.
Categorical feature	Mode / explicit category	Preserves categorical representation when justified.
Complex patterns	KNN/regression/interpolation	Can use relationships in the data; requires more care.
Very high missingness	Consider dropping feature	Only if the feature contributes little or cannot be recovered.

For predictive ML, the safest implementation is usually to put imputation inside a pipeline so the imputer learns its statistic from training data only.

4. Duplicate Rows

Exact duplicates can distort frequency-based analysis and may give a model repeated influence from the same observation. But repeated records are not automatically errors: some datasets legitimately contain multiple identical-looking events.

```
duplicate_count = df.duplicated().sum()
df = df.drop_duplicates().reset_index(drop=True)
```

5. Data-Type and Format Cleaning

Common issues include numbers stored as strings, dates stored as text, leading/trailing spaces and mixed representations.

```
df["age"] = pd.to_numeric(df["age"], errors="coerce")
df["date"] = pd.to_datetime(df["date"], errors="coerce")

# Text normalization
df["city"] = df["city"].str.strip().str.lower()
```

6. Inconsistent Categories

Category inconsistencies are common in real data. Before encoding, normalize spelling, whitespace and capitalization, then verify the resulting categories.

```
# Example
mapping = {
    "male": "male",
    "Male": "male",
    "MALE": "male",
    "m": "male"
}
df["gender"] = df["gender"].replace(mapping)

print(df["gender"].value_counts(dropna=False))
```

7. Outliers — Detection and Treatment

The PDF mentions boxplots, IQR and z-score approaches. The important point is that an outlier is an observation that deserves investigation, not automatically an observation that must be deleted.

IQR rule: $Q1 = 25\text{th percentile}$, $Q3 = 75\text{th percentile}$, $IQR = Q3 - Q1$. A common rule flags values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$.

```
Q1 = df["income"].quantile(0.25)
Q3 = df["income"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["income"] < lower) | (df["income"] > upper)]
```

Possible treatments: remove clearly invalid records, cap extreme values when appropriate, transform the feature, or retain genuine extreme observations.

8. Domain and Logic Validation

Statistical checks are not enough. A value can be statistically unusual but valid, or statistically ordinary but logically impossible. Use domain rules whenever they are available.

```
# Example domain checks
bad_age = df[(df["age"] < 0) | (df["age"] > 120)]

# Example relationship rule
bad_experience = df[df["experience"] > df["age"]]
```

9. Cleaning vs Preprocessing

Cleaning asks: Is this record valid and internally consistent?

Preprocessing asks: How should valid data be represented so a model can use it?

10. Encoding Categorical Variables

Ordinal/label encoding: appropriate when categories have meaningful order, such as Low < Medium < High.

One-hot encoding: appropriate for nominal categories such as city/region where numerical ordering has no meaning.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
X_encoded = encoder.fit_transform(df[["city"]])
```

Avoid arbitrary numeric labels for nominal categories if the model could interpret the numbers as ordered.

11. Feature Transformation

Transformations can make a feature more suitable for a model or easier to work with. The PDF mentions logarithm and square-root transformations for skewed data.

```
import numpy as np

# For positive/right-skewed values
df["income_log"] = np.log1p(df["income"])
```

Always check the resulting distribution and the model's requirements before deciding that a transformation is useful.

12. Feature Scaling

Scaling puts numerical features onto comparable scales. The PDF specifically describes Min-Max normalization and Z-score standardization and notes their usefulness for distance-based algorithms.

Min-Max: $x' = (x - \min) / (\max - \min)$, usually producing values between 0 and 1.

Standardization: $z = (x - \mu) / \sigma$, producing a feature centered around 0 with standard deviation near 1.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

minmax = MinMaxScaler()
standard = StandardScaler()

X_minmax = minmax.fit_transform(X_train)
X_standard = standard.fit_transform(X_train)
```

Do not fit a scaler on the test set. Fit on training data, then transform both training and test data.

13. Feature Engineering

Feature engineering creates new features or transforms existing ones to expose useful patterns. The PDF lists mathematical combinations, target-based flags, binning and time-based features as common techniques.

```
# Example
df["income_per_experience"] = df["income"] / (df["experience"] + 1)

# Example flag
df["senior"] = (df["experience"] >= 10).astype(int)
```

Feature engineering should be driven by the problem/domain and validated empirically. Do not create features that leak the target.

14. Feature Selection

Feature selection means retaining useful features and removing irrelevant/redundant ones. The PDF highlights benefits such as reducing noise/overfitting, speeding training and improving interpretability.

Method	Example	Typical idea
Filter	Correlation	Remove highly redundant numerical features.
Filter	Chi-square	Categorical feature vs categorical target.

Filter	ANOVA F-test	Numerical feature vs categorical target.
Embedded	Lasso	Regularization can shrink some coefficients toward zero.
Embedded	Tree models	Use model-derived feature importance.

15. End-to-End Practical Pipeline

A robust workflow separates exploration from the reusable transformation pipeline.

```
# 1. Load data
df = pd.read_csv("data.csv")

# 2. Understand data
df.head()
df.info()
df.describe(include="all")

# 3. EDA
# missingness, duplicates, distributions, relationships, target analysis

# 4. Split
X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 5. Build preprocessing
numeric_features = [...]
categorical_features = [...]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])

# 6. Fit preprocessing only on training data
X_train_ready = preprocessor.fit_transform(X_train)
X_test_ready = preprocessor.transform(X_test)

# 7. Then connect a model and evaluate it.
```

16. Data Leakage — The Rule to Remember

Data leakage happens when information that should be unavailable during training influences the training process. A common source is fitting preprocessing transformations using the complete dataset before the train/test split.

Correct: split → fit preprocessing on train → transform train/test → train model → evaluate on test.

Incorrect: fit imputer/scaler/feature selection on all data → split → train/evaluate.

17. EDA vs Cleaning vs Preprocessing — Quick Comparison

Stage	Main question	Examples
EDA	What does the data look like?	shape, stats, plots, missingness, relationships
Cleaning	What is wrong with the data?	missing values, duplicates, types, categories, invalid values
Preprocessing	How should valid data be represented for ML?	encoding, scaling, transformations
Feature engineering	Can we expose better signals?	ratios, flags, bins, time features
Feature selection	Which features should remain?	correlation, chi-square, ANOVA, Lasso, tree importance

18. Final Checklist

- Understand the problem and target before touching transformations.
- Inspect structure, types and basic statistics.
- Quantify missing values and duplicates.
- Explore distributions and category frequencies.
- Investigate outliers instead of automatically deleting them.
- Validate domain rules and logical relationships.
- Standardize inconsistent categories.
- Split train/test before fitting learned preprocessing.
- Encode categorical variables appropriately.
- Scale numerical variables when the chosen model benefits from it.
- Transform skewed variables only when justified.
- Engineer domain-relevant features without target leakage.
- Select features using methods appropriate to the task.
- Only after this preparation move to model selection, training and evaluation.

Source mapping

This in-depth document expands the topics shown in the uploaded 60-page PDF. The PDF's page 30 gives the 11-stage ML workflow; pages 33–39 cover EDA steps; pages 40–47 cover data cleaning; pages 48–55 cover preprocessing; and pages 56–60 cover feature engineering and feature selection. The additional implementation detail here is a practical expansion for Python/ML study rather than a claim that every code example appears verbatim in the source.